

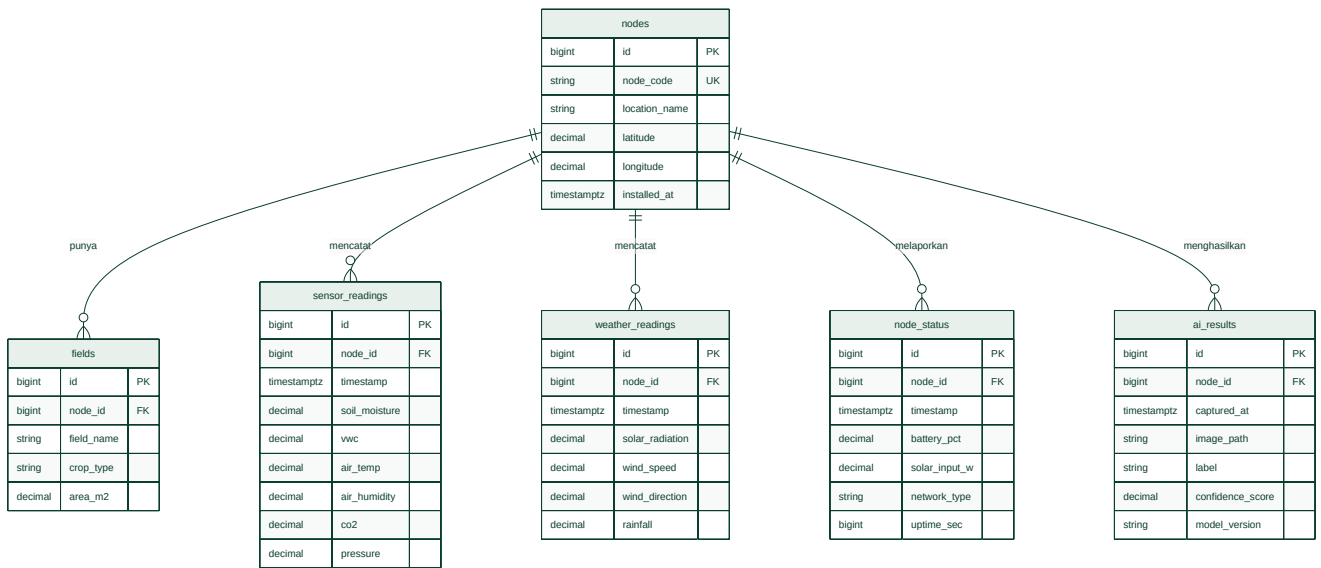
S-AGRI — Arsitektur Database & Panduan Integrasi Sensor

Dokumen ini menjelaskan **struktur database** S-AGRI Digital Twin dan **cara perangkat sensor mengirim data** ke sistem lewat REST API. Ditujukan untuk developer yang membangun node sensor (mis. ESP32/Raspberry Pi) maupun backend.

- **Base URL API:** `https://sagri.my.id/api`
- **Format:** JSON. Selalu kirim header `Accept: application/json` (dan `Content-Type: application/json` untuk POST).
- **Autentikasi:** endpoint data (POST `sensor/cuaca/status/AI`) saat ini **publik** (tanpa token) demi kemudahan integrasi awal. Jangan menaruh data sensitif; token auth bisa ditambahkan nanti.
- **Waktu:** semua kolom waktu bertipe `timestampz`. Kirim timestamp dalam **ISO 8601 dengan offset**, disarankan UTC, contoh: `2026-07-24T02:00:00Z`.
- **WAJIB** kirim header **User-Agent**: situs ada di belakang Cloudflare, yang **memblokir User-Agent default beberapa library** (mis. `Python-urllib`) dengan **error 1010 "Access denied"**. Selalu setel `User-Agent` custom apa pun (mis. `S-AGRI-Sensor/1.0`). `curl` dan `ESP32 HTTPClient` sudah aman secara default; `python-urllib` diblokir. Kalau lupa, POST/GET akan balas **HTTP 403** (bukan 201/200).

1. Arsitektur Database

Satu **node** = satu perangkat/stasiun sensor di lapangan. Setiap node punya banyak pembacaan yang bertambah seiring waktu (time-series): sensor tanah/udara, cuaca, status perangkat, dan hasil inferensi AI. Satu node juga bisa menaungi satu/lebih **field** (lahan/petak).



Relasi: `fields`, `sensor_readings`, `weather_readings`, `node_status`, dan `ai_results` semuanya belongsTo sebuah `node` lewat `node_id` (foreign key, **cascade on delete** — hapus node → semua datanya ikut terhapus).

Tabel `nodes` — master perangkat

Kolom	Tipe	Wajib	Keterangan
id	bigint (PK)	auto	ID internal — dipakai sebagai <code>node_id</code> di semua tabel lain

Kolom	Tipe	Wajib	Keterangan
node_code	string (unik)	✓	Kode unik perangkat, mis. NODE-RICE-01
location_name	string	✓	Nama lokasi, mis. Main rice field – Yilan
latitude	decimal(10,7)	✓	-90..90
longitude	decimal(10,7)	✓	-180..180
installed_at	timestamptz	–	Tanggal pemasangan
created_at / updated_at	timestamptz	auto	

Tabel `fields` — lahan/petak

Kolom	Tipe	Wajib	Keterangan
id	bigint (PK)	auto	
node_id	bigint (FK)	✓	Mengacu ke <code>nodes.id</code>
field_name	string	✓	mis. Main Field of Rice
crop_type	string	–	mis. rice , green_onion (dipakai dashboard untuk memetakan crop → node)
area_m2	decimal(12,2)	–	Luas lahan (m²)

Tabel `sensor_readings` — sensor tanah & udara (time-series)

Kolom	Tipe	Satuan	Keterangan
node_id	bigint (FK)	–	Wajib
timestamp	timestamptz	–	Waktu pembacaan (wajib)
soil_moisture	decimal(8,3)	%	Kelembapan tanah
vwv	decimal(8,3)	%	Volumetric Water Content
air_temp	decimal(6,2)	°C	Suhu udara
air_humidity	decimal(6,2)	%	Kelembapan udara
co2	decimal(10,2)	ppm	Kadar CO ₂
pressure	decimal(8,2)	hPa	Tekanan udara

Semua kolom metrik **opsional** (nullable) — kirim yang tersedia saja.

Tabel `weather_readings` — stasiun cuaca (time-series)

Kolom	Tipe	Satuan	Keterangan
node_id	bigint (FK)	–	Wajib
timestamp	timestamptz	–	Wajib
solar_radiation	decimal(10,3)	W/m²	Radiasi matahari
wind_speed	decimal(8,3)	m/s	Kecepatan angin
wind_direction	decimal(6,2)	derajat	Arah angin (0–360)

Kolom	Tipe	Satuan	Keterangan
rainfall	decimal(8,3)	mm	Curah hujan

Tabel `node_status` — kesehatan perangkat (time-series)

Kolom	Tipe	Satuan	Keterangan
node_id	bigint (FK)	–	Wajib
timestamp	timestampz	–	Wajib
battery_pct	decimal(5,2)	%	Level baterai (0–100)
solar_input_w	decimal(8,3)	W	Daya masuk panel surya
network_type	string	–	mis. 4G , LoRa , WiFi
uptime_sec	bigint	detik	Lama perangkat menyala

Tabel `ai_results` — hasil inferensi AI (deteksi penyakit daun)

Kolom	Tipe	Keterangan
node_id	bigint (FK)	Wajib
captured_at	timestampz	Waktu gambar diambil (wajib)
image_path	string	Path/URL gambar (wajib)
label	string	Label hasil, mis. healthy , leaf_blast
confidence_score	decimal(6,4)	Keyakinan model, 0–1
model_version	string	Versi model, mis. yolov8-sagri-v1.3

2. Alur Integrasi Sensor

(sekali saat setup)

1. Daftarkan node
(POST /nodes)

2. (opsional) buat field
(POST /fields)

(berulang, periodik)

3. Kirim sensor_readings (mis. tiap 1–5 menit)

4. Kirim weather_readings (mis. tiap jam)

5. Kirim node_status (mis. tiap jam)

6. (opsional) kirim ai/result

Langkah 1 — daftarkan node satu kali, catat `id` yang dikembalikan. `id` inilah `node_id` yang dipakai di semua pengiriman data berikutnya. (Kalau node sudah ada, ambil `id` -nya via `GET /api/nodes`.)

3. Referensi Endpoint (POST — mengirim data)

Konsep inti: satu jenis data = satu endpoint = satu tabel. Tidak ada satu endpoint tunggal untuk semua data. Setiap jenis data dikirim ke endpoint-nya masing-masing, yang menulis ke tabel-nya sendiri. Semuanya terhubung ke perangkat lewat `node_id` — jadi setiap pengiriman **wajib** menyertakan `node_id` yang valid.

Peta endpoint ↔ tabel

Jenis data	POST (kirim)	Tabel tujuan	GET (baca)
Identitas perangkat	/api/nodes	nodes	/api/nodes
Lahan/petak	/api/fields	fields	/api/fields
Sensor tanah & udara	/api/sensors/data	sensor_readings	/api/sensors/latest , /api/sensors/history
Cuaca	/api/weather-readings	weather_readings	/api/weather/latest , /api/weather/history
Status perangkat	/api/nodes/status	node_status	/api/nodes/status
Hasil AI	/api/ai/result	ai_results	/api/ai/latest , /api/ai-results

Alur pakai: **daftar perangkat sekali** (POST /api/nodes → dapat node_id), lalu tiap perangkat **kirim berulang** ke endpoint yang sesuai jenis datanya sambil selalu membawa node_id -nya.

Semua respons sukses = **HTTP 201** berisi baris yang tersimpan (termasuk id). Kegagalan validasi = **HTTP 422** berisi { "message": ..., "errors": { field: [pesan] } } .

3.1 Daftarkan node — POST /api/nodes

Field	Wajib	Aturan
node_code	✓	string, unik
location_name	✓	string
latitude	✓	numeric, -90..90
longitude	✓	numeric, -180..180
installed_at	—	date (ISO 8601)

```
curl -X POST https://sagri.my.id/api/nodes \
  -H "Content-Type: application/json" -H "Accept: application/json" \
  -d '{
    "node_code": "NODE-RICE-02",
    "location_name": "Experimental paddy – Yilan",
    "latitude": 24.7512,
    "longitude": 121.7530,
    "installed_at": "2026-07-24T00:00:00Z"
  }'
# → 201 {"id": 7, "node_code": "NODE-RICE-02", ...} ← simpan id = 7
```

3.2 (Opsional) Buat field — POST /api/fields

Field	Wajib	Aturan
node_id	✓	harus ada di tabel nodes
field_name	✓	string
crop_type	—	string (mis. rice , green_onion)
area_m2	—	numeric ≥ 0

3.3 Kirim pembacaan sensor — POST /api/sensors/data

Field	Wajib	Aturan / Satuan
node_id	✓	harus ada di tabel nodes
timestamp	✓	date ISO 8601
soil_moisture	—	numeric, %
vwc	—	numeric, %
air_temp	—	numeric, °C
air_humidity	—	numeric, %
co2	—	numeric, ppm
pressure	—	numeric, hPa

```
curl -X POST https://sagri.my.id/api/sensors/data \
-H "Content-Type: application/json" -H "Accept: application/json" \
-d '{
  "node_id": 7,
  "timestamp": "2026-07-24T02:00:00Z",
  "soil_moisture": 81.4,
  "vwc": 52.0,
  "air_temp": 26.3,
  "air_humidity": 84.1,
  "co2": 415.2,
  "pressure": 1012.4
}'
```

3.4 Kirim data cuaca — POST /api/weather-readings

Field	Wajib	Aturan / Satuan
node_id	✓	harus ada di tabel nodes
timestamp	✓	date ISO 8601
solar_radiation	—	numeric, W/m ²
wind_speed	—	numeric, m/s
wind_direction	—	numeric, derajat
rainfall	—	numeric, mm

3.5 Kirim status perangkat — POST /api/nodes/status

Field	Wajib	Aturan / Satuan
node_id	✓	harus ada di tabel nodes
timestamp	✓	date ISO 8601
battery_pct	—	numeric, 0..100

Field	Wajib	Aturan / Satuan
solar_input_w	–	numeric, W
network_type	–	string (4G / LoRa / WiFi)
uptime_sec	–	integer ≥ 0

3.6 Kirim hasil AI — `POST /api/ai/result`

Field	Wajib	Aturan
node_id	✓	harus ada di tabel nodes
captured_at	✓	date ISO 8601
image_path	✓	string
label	–	string
confidence_score	–	numeric, 0..1
model_version	–	string

4. Endpoint Baca (GET — untuk verifikasi/dashboard)

Endpoint	Keterangan
<code>GET /api/nodes</code>	Daftar node (paginasi)
<code>GET /api/fields</code>	Daftar field (paginasi)
<code>GET /api/sensors/latest?node_id={id}</code>	Pembacaan sensor terbaru sebuah node
<code>GET /api/sensors/history?node_id={id}&range=1d 7d 30d</code>	Riwayat sensor
<code>GET /api/weather/latest?node_id={id}</code>	Cuaca terbaru
<code>GET /api/weather/history?node_id={id}&range=1d 7d 30d</code>	Riwayat cuaca
<code>GET /api/nodes/status?node_id={id}</code>	Status perangkat terbaru
<code>GET /api/ai/latest?node_id={id}</code>	Hasil AI terbaru
<code>GET /api/ai-results?node_id={id}</code>	Riwayat hasil AI (paginasi, terbaru dulu)
<code>GET /api/export/sensors?node_id={id}</code>	Ekspor pembacaan sensor sebagai CSV

5. Contoh Kode

5.1 Python (requests) — cocok untuk Raspberry Pi / gateway

```
import requests
from datetime import datetime, timezone

BASE = "https://sagri.my.id/api"
NODE_ID = 7 # dari langkah 3.1
```

```

payload = {
    "node_id": NODE_ID,
    "timestamp": datetime.now(timezone.utc).isoformat(),
    "soil_moisture": 81.4,
    "vwc": 52.0,
    "air_temp": 26.3,
    "air_humidity": 84.1,
    "co2": 415.2,
    "pressure": 1012.4,
}

# User-Agent WAJIB – tanpa ini Cloudflare balas 403 (error 1010).
headers = {"Accept": "application/json", "User-Agent": "S-AGRI-Sensor/1.0"}
r = requests.post(f"{BASE}/sensors/data", json=payload, headers=headers, timeout=10)
print(r.status_code, r.json())
if r.status_code == 422:
    print("Validasi gagal:", r.json()["errors"])

```

5.2 ESP32 / Arduino (HTTPClient) — kirim langsung dari mikrokontroler

```

#include <WiFi.h>
#include <HTTPClient.h>

const char* API = "https://sagri.my.id/api/sensors/data";
const int NODE_ID = 7;

void postReading(float temp, float hum, float soil) {
    if (WiFi.status() != WL_CONNECTED) return;
    HTTPClient http;
    http.begin(API); // gunakan WiFiClientSecure utk sertifikat penuh
    http.addHeader("Content-Type", "application/json");
    http.addHeader("Accept", "application/json");
    http.addHeader("User-Agent", "S-AGRI-Sensor/1.0"); // hindari blokir Cloudflare

    // Idealnya timestamp dari RTC/NTP dalam format ISO 8601 UTC:
    String body = "{";
    body += "\"node_id\": " + String(NODE_ID) + ",";
    body += "\"timestamp\": \"2026-07-24T02:00:00Z\",";
    body += "\"air_temp\": " + String(temp, 2) + ",";
    body += "\"air_humidity\": " + String(hum, 2) + ",";
    body += "\"soil_moisture\": " + String(soil, 2);
    body += "}";

    int code = http.POST(body);
    Serial.printf("HTTP %d: %s\n", code, http.getString().c_str());
    http.end();
}

```

5.3 Simulator siap pakai (untuk tes sebelum hardware ada)

Repo ini menyertakan `tools/sensor_simulator.py` — mengirim data realistis ke API secara periodik, tanpa dependency (cukup Python 3):

```
python tools/sensor_simulator.py --node-id 5          # kirim tiap 15 detik
python tools/sensor_simulator.py --node-id 5 --interval 5 # tiap 5 detik
python tools/sensor_simulator.py --register           # buat node baru dulu
python tools/sensor_simulator.py --node-id 5 --count 3  # 3x lalu berhenti (tes cepat)
```

Pakai ini untuk memverifikasi seluruh alur (API → PostgreSQL → dashboard) berjalan sebelum node sensor fisik siap.

6. Tips & Catatan Penting

- **Ambil `node_id` sekali, lalu hardcode/simpan** di perangkat. Semua pengiriman butuh `node_id` yang valid; kalau salah → error 422 "The node id field ... exists".
 - **Timestamp:** pakai jam yang tersinkron (NTP/RTC). Format ISO 8601 UTC (`...Z`) paling aman. Jangan kirim timestamp lokal tanpa offset.
 - **Nilai desimal:** perhatikan presisi kolom (mis. `air_temp = decimal(6,2)` → maksimal 2 angka di belakang koma; `co2 = decimal(10,2)`). Nilai di luar rentang presisi bisa ditolak/dibulatkan.
 - **Kirim sebagian field juga boleh** — kolom metrik bersifat opsional; kirim yang sensornya tersedia.
 - **Batasi frekuensi** sesuai kebutuhan (mis. sensor tiap 1–5 menit, cuaca & status tiap jam) agar hemat daya & bandwidth. Untuk banyak pembacaan sekaligus, kirim satu per satu ke endpoint (belum ada endpoint batch).
 - **Penanganan error:** cek HTTP status. 201 = sukses. 422 = ada field salah (lihat `errors`). 5xx = masalah server — coba ulang dengan backoff.
 - **Verifikasi:** setelah kirim, cek `GET /api/sensors/latest?node_id={id}` — harusnya menampilkan data yang baru dikirim.
-

Backend: Laravel (PostgreSQL) di `sagri.my.id`. Untuk pertanyaan skema/endpoint, lihat `routes/api.php` dan `app/Http/Controllers/Api/` di repo ini.